

LingBot-VLA 2.0 的 LeRobot 移植: 代码逻辑、上游适配与框架适配

给内部同学的代码导读——这套移植怎么读、上游权重怎么来的、怎么长在 lerobot 里

LingBot-VLA 移植工作组 代码:lingbot-lerobot-final(私有) 主干 2902c42 上游
PR:huggingface/lerobot #3967

摘要 本文是 LingBot-VLA 2.0 LeRobot 移植的代码交接导读, 面向要通读并 review 这套代码的内部同学。全文回答三个问题:(1) **代码逻辑**——双流架构 (Qwen3-VL-4B 主干 + 36 层稀疏 MoE 动作专家) 如何组织成 lerobot 的 BYOP 三件套, 一次推理的前向三阶段 (前缀编码 → KV 复用去噪 → 反归一化) 各做什么;(2) **上游适配**——官方发布的 robblyant/lingbot-vla-v2-6b 权重为何不能直接加载、一次性转换器做了什么、与 transformers 原生实现的逐位数值一致性如何;(3) **框架适配**——策略如何零特判地接入 lerobot 的 processor 注册链、训练/推理入口与多卡路线。附一页性能速查与已知边界。数值上与上游逐位等价, CI 全绿, 唯一未闭环项为 RoboTwin 闭环 port 级复评。

关键词: VLA; LingBot-VLA 2.0; lerobot BYOP; 流匹配; 稀疏 MoE; checkpoint 转换; 逐位一致性

1 一页总览

是什么。LingBot-VLA 2.0 是 pi0 式双流 VLA: Qwen3-VL-4B-Instruct 作视觉-语言主干, 并联一个 36 层 (hidden 768) 的 Qwen2 动作专家, 其**全部 36 层**均为稀疏 MoE (32 专家、top-4 路由、专家中间维 512、共享专家 704); 动作由流匹配生成, 默认 10 步 Euler 去噪, 一次输出 chunk 50。全模型 6.23B 参数。本移植将其以 lerobot BYOP (Build-Your-Own-Policy) 形式落为策略 lingbot_vla_v2。

交付态 (截至 2026-08-26):

- huggingface/lerobot PR #3967: 四轮 review 意见**全部清零** (细节见仓库内 PR3967_REVIEW_TODO_20260826.md), 全绿 (pre-commit 19 hooks + pytest 422 passed / 18 skipped);
- 私有存档库 miracle-techlink/lingbot-lerobot-final (单提交快照, 即本次 review 对象);
- 数值一致性: 与 transformers 4.57 原生前向**逐位等价** (唯一残差为 triton fused_moe kernel 内部 MMA ≈ 0.06 , 良性)。

端到端数据流 (一次 predict_action_chunk):

```

obs(相机 ×N / state / task 文本)
↓ PreProcessor:tokenizer + Qwen 图像预处理 (GPU 可选)+
  FeatureTransform.apply(slot 映射 + 归一化)
↓ embed_prefix:vision tower 编码图像 + 文本 → 36 层前缀 KV cache(每次推理仅一次)
↓ 去噪环 × num_steps(默认 10):predict_velocity 逐步走专家流——每层注意力复用前缀
  KV + MoE FFN
↓ 速度场 ODE 积分 → action chunk(1, 50, action_dim)
↓ PostProcessor:FeatureTransform.unapply(canonical → 机器人关节空间, 反归一化; 缺
  配置即 raise)

```

2 代码逻辑

2.1 文件地图

代码在 `src/lerobot/policies/lingbot_vla_v2/` 下, 按 lerobot BYOP 惯例组织:

表 1: 文件职责一览 (review 按此表定位)

文件	职责
<code>configuration_*.py</code>	策略配置 (draccus 注册, type: lingbot_vla_v2)。resolve_robot_config_and_stats 做”显式路径优先、内嵌内容兜底”的资产解析; build_feature_transform_configs 是 apply/unapply 两侧共用的单一配置来源; save_pretrained 在保存时清空机器人本地路径字段, 保证 checkpoint 自包含
<code>modeling_*.py</code>	LingbotVLAV2Policy 与内部 LingBotVLAFlowMatchingModel: 前缀编码、KV cache 生命周期、去噪环、_postprocess_actions 反归一化 (反归一化器构建失败直接 raise, 绝不静默输出归一化值——真机上等于任意关节命令)
<code>processor_*.py</code>	ProcessorStep 管线: pre(tokenize + Qwen 图像预处理 + DeviceProcessorStep)/ post(unapply); 提供 make_*_pre_post_processors_from_pretrained, 支持 config 派生的 feature_step_overrides 注入
<code>feature_transform.py</code>	FeatureTransform: 55 维 canonical 空间 (max_state_dim/max_action_dim 定长 padding) 的 slot 映射、逐 slot 归一化统计、apply/unapply 双向
<code>qwen3vl_in_vla.py</code>	vendored Qwen3-VL 塔 + VLM 解码层。改造点: 双流 forward 与前缀 KV 注入 (这是 transformers 里没有的部分)
<code>qwen2_action_expert.py</code>	vendored Qwen2 动作专家层: 含 fused 布局 MoE 与小 batch 稠密双 GEMM 路径
<code>convert_upstream_checkpoint.py</code>	一次性转换器 (见 §3)
<code>configuration_*_internal.py</code>	上游兼容配置 (architectures 指向 FlowMatching / FlowMatchingV2)

2.2 前向三阶段

(1) **前缀编码** (embed_prefix, 每次推理一次): 图像经 vision tower 得视觉 token(grid-thw 可由 precompute_grid_thw 预计算缓存), 与文本 token 拼接, 过 VLM 主干各层, 把 36 层的 K/V 写入前缀 cache。此后整个去噪过程不再触碰主干权重。

(2) **去噪环** (predict_velocity × num_steps): 动作侧只有 51 个 suffix token(state 1 + action chunk 50)。每步把带噪动作嵌入送入专家流, 每层注意力复用前缀 KV(cross-attention 语义, 但实

现为同一序列的前缀后缀拼接)、FFN 走 MoE top-4; 得到速度场后 Euler 积分。步数是质量-延迟旋钮 (4/7/10 实测见 §5)。

(3) 反归一化: 输出从 55 维 canonical 空间经 unapply 回到具体机器人关节空间并反归一化。apply 与 unapply 共用 build_feature_transform_configs 生成的同一份配置——两侧若各自手写会漂移 (actions 在 A 布局下生成、B 布局下反归一化), 这是 review 抓出并修掉的真实风险点。

2.3 关键机制速记

- **MoE 两条路**: checkpoint 原生 fused(按专家维堆叠) 布局; 当 $B \times T \leq 512$ 时自动走稠密双 GEMM 纯 PyTorch 路径——两个 GEMM 算完 32 专家、top-4 权重折叠进 down GEMM, 无路由 kernel、静态形状、dynamo 可整图追踪 (与精确路由代数等价, fp32 相对差 $1.4e-6$)。
- **attention 三实现**: eager / sdpa / flex (flex $\rightarrow$ triton 编译)。训练推荐 sdpa+bf16; 推理 compile 下 eager 与 sdpa 等价; fa2 在上游即 NotImplementedError。
- **加速旋钮** (默认关): use_cudagraph_denoise (CUDA graph 捕去噪环, 空 cache 捕获)、preprocess_device=cuda、compile_predict_velocity。终烘配方见 §5。

3 上游适配: robbyant 权重 $\rightarrow$ LeRobot checkpoint

为什么必须转换。官方发布物 robbyant/lingbot-vla-v2-6b 的 config.json 只有一行 {"vlm_family": "qwen3_v1"}——全部架构信息住在官方训练脚本里, 任何标准加载器都无法重建模型。早期版本用过“运行时 sniffer + raw loader”, 但 reviewer 正确指出这会把 4 个特判分支打进 lerobot 核心文件, 且 sniffer 会误认别人的 checkpoint; 最终方案是一次性转换 + 零特判。

转换器做什么 (convert_upstream_checkpoint.py, 跑一次):

- 重打 config.json: type: lingbot_vla_v2 + 全部结构 override——从此走标准 from_pretrained, 多卡加载自动带 #4042 修复;
- 权重切分: 1708 张量 $\rightarrow$ **1624 加载 + 84 显式剔除** (depth 对齐 / 视频蒸馏头, use_depth=False; 在转换期一次剔除并记录, 而非每次加载静默跳过); strict 加载 0 missing / 0 unexpected;
- 内嵌机器人资产: robot_config 与 norm_stats 内容写进 config 与 processor JSON, 保存时清空路径字段 $\Rightarrow$ checkpoint 拷到任何机器都能重载 (自包含)。

数值一致性 (转换正确性的证据): 同权重同输入同噪声, transformers 4.57 原生实现 vs 本移植, 前缀 embeddings、全层 KV cache、suffix/时间嵌入、MoE router logits、top-4 选中专家全部 $\max|\Delta|=0$ (bf16 与 fp32 逐位一致)。唯一残差在 vendored triton fused_moe 的 SwiGLU 内部 (≈ 0.06 /块), 属 kernel 内部低精度 MMA 的良性差异, 非结构或加载缺陷。

与 transformers 的复用边界: 能用 transformers 的 (Qwen3-VL 配置与构件、Qwen2 层、tokenizer/processor) 直接用; 留在 LeRobot 的是上游没有的部分——双流 forward、前缀 KV 注入、流匹配动作头、MoE 专家逻辑。vendored 文件顶部有来源标注。

4 LeRobot 框架适配

BYOP 合规: policies/lingbot_vla_v2/ 三件套 (configuration / modeling / processor) + README.md 软链到 docs/source/policy_lingbot_vla_v2_README.md + lingbot_vla_v2.mdx (含 Benchmark 章节与复现命令) + 根 README 模型表 + modelcard 模板条目。

Processor 体系接入:

- ProcessorStep 注册走包导入链: 包内 __init__.py 导出 configuration/modeling/processor

三模块 (与其他 21 个策略一致),import lerobot.policies 即注册全部 60 个 step, 含 lingbot_vla_2_feature_transform——factory 里为此曾存在的通用 importlib 补丁块已删除;

- 管线经 PolicyProcessorPipeline.from_pretrained 加载;config 派生的 feature_step_overrides 注入, 使新本体 fine-tune 能覆盖 checkpoint 里的槽映射;
- lerobot 通用 normalizer override 在此被过滤——本策略归一化在 FeatureTransform 内, 不另设 normalizer step(过滤逻辑在 lingbot processor 模块内, 不污染共享代码)。

零 特 判:configs/policies.py、factory.py、lerobot_train.py、lerobot_eval.py 四个核心文件中 lingbot 特判为 0 处;processor 工厂按 isinstance 分发 (与 Groot/Evo1 同款);_no_split_modules 登记真实层类名,FSDP TRANSFORMER_BASED_WRAP 开箱即用。

训练 / 推理入口:

入口	路线	状态
lerobot-train	FSDP1	8×A100 干净起训 (5.8B/6.2B 可训练,loss ~0.6)
	FSDP2	2×4090 冒烟通过 (auto-wrap + offload)
	DDP	2→4 卡线性扩展,8 卡 30k 步 ≈ 9–11 h
	LoRA	社区 PEFT 三钩子路线, 单卡 12.4 GB, 冒烟通过
lerobot-eval / lerobot-rollout	标准	predict_action_chunk 返回 (1, 50, 6), 数据集单位

5 性能速查与已知边界

表 2: 性能速查 (详报:lingbot_vla_v2_total_report.pdf 与 ~/reports/ 汇总)

项	数字	对照
数值一致性	1624 keys 0/0; 前缀/KV/router 全逐位	transformers 4.57 原生
训练 e2e(A100, B=2)	2378 → 1137 ms(-52%)	PR 原版 → sdpa+bf16+fused AdamW
训练吞吐	B=3 1.99 samples/s ;LoRA 12.4 GB/卡	A100-80G
推理 e2e(A100)	1705 → 436.6 ms(3.9×)	官方 deploy eager → 本移植优化栈
推理 (4090, 终烘)	130.17 ms (7 步 +CUDA graph)	官方 130 ms 口径复现;4/7/10 步 = 108.5/130.9/153.0
推理 (GB10)	635 / 752 ms(4 步/7 步)	带宽地板:273 GB/s,130ms 物理不可达
真机 RTC 云端推理	60 s 49 次 0 错;RTT p50 822 → 630 ms	q75 量化 + threshold 30 + horizon 24

已知边界 (review 时请知悉):

- **RoboTwin 闭环 port 级复评未完成:** 官方权重上游成绩 Clean 93.52% / Randomized 92.80%; 本移植的闭环初评 0/12 已定位为评测基建问题 (评测 ckpt 内嵌慢配置), 待快配置重导出后重评——这是当前唯一未闭环的验证项;
- depth / 视频蒸馏分支在本 checkpoint 不可达 (use_depth=False,84 张量已显式剔除), 如需启用走 follow-up;
- reviewer 认可延后的 8 项 cosmetic follow-up(process-wide transformers patch、asserts 收敛、license 头、文件重组等) 见 TODO 文档 queued 一节;
- 四轮 PR review 已全部清零, 逐条对照表:PR3967_REVIEW_TODO_20260826.md(仓库根)。

附录: 代码位置速查

想看什么	去哪
双流前向 / KV 注入	modeling_lingbot_vla_v2.py(LingBotVLAFlowMatchingModel)+ qwen3vl_in_vla.py
去噪环 / 速度场	modeling 中 predict_velocity;CUDA graph 路径同文件
图像 / 文本预处理	processor_lingbot_vla_v2.py(管线)+ feature_transform.py(apply)
归一化 / slot 映射	feature_transform.py(FeatureTransform.apply/unapply)
MoE 专家实现	qwen2_action_expert.py(fused 布局 + 稠密双 GEMM)
checkpoint 转换	convert_upstream_checkpoint.py
训练配方 / 4090 配置	docs/source/lingbot_vla_v2.mdx;bench/robotwin/
外部指针	私有库 lingbot-lerobot-final;PR #3967;~/reports/lingbot-vla-v2-训练 推理评测汇总-20260826.md;加速详报 docs/accel/lingbot_vla_v2_total_report.pdf